

state) of the switch was represented by a single Boolean variable; `True` represented on and `False` represented off.

There are many situations where a program or an object can be in one of a number of different states, and different code needs to run based on the current state. For example, consider the series of steps involved in using an ATM. There is a starting (greeting) state, then you need to put in your ATM card; after this you're prompted to enter your PIN, choose which action you want to do, and so on. At any point, you may need to go back a step or even start over. The general implementation approach is to use a *state machine*.

state machine

A model that represents and controls the flow of execution through a series of states.

The implementation of a state machine consists of:

- A collection of predefined states, typically expressed as constants whose values are strings comprised of a word or short phrase that describes what happens in the state
- A single variable to track the current state
- A starting state (from the set of predefined states)
- A set of clearly defined transitions between states

A state machine can only be in one state at any given time, but can move to a new state, typically based on specific input from the user.

In Chapter 7 we discussed the GUI button classes in the `pygwidgets` package. When rolling over and clicking a button, a user sees three different images—up, over, and down—which correspond to different states of the button. The image switching is done in the `handleEvent()` method (which is called whenever an event occurs). Let's take a closer look at how this is implemented.

The `handleEvent()` method is built as a state machine. The state is kept in an instance variable, `self.state`. Each button starts in the up state, showing the